



# LABORATORIO DI RETI INFORMATICHE

CORSO DI LAUREA TRIENNALE IN INGEGNERIA INFORMATICA  
A.A. 2025/2026

Francesca Righetti  
[francesca.righetti@unipi.it](mailto:francesca.righetti@unipi.it)



# ESERCITAZIONE 6

Programmazione con i socket I –  
Esercizi di programmazione distribuita

# Installare GCC

## ■ GCC: GNU Compiler Collection

- Compilatore per C, C++
- Interfaccia a riga di comando
- Comando per installarlo: `sudo apt install gcc`

# Leggere gli argomenti

```
int main(int argc, char* argv[]){  
    /* ... */  
}  
int main(int argc, char** argv){  
    /* ... */  
}
```

- **argc** è il numero degli argomenti passati + 1
- **argv** è un array di stringhe
  - Quindi un puntatore a puntatori di caratteri
  - La prima stringa rappresenta il comando
  - Se ci servono valori numerici bisogna convertire le stringhe (atoi(), atol(), ...)

# Leggere gli argomenti

```
int main(int argc, char* argv){  
    int i;  
    printf("Ci sono %d argomenti:\n", argc);  
    for (i = 0; i < argc; ++i) {  
        printf("%s\n", argv[i]);  
    }  
    return 0;  
}
```

```
$ ./mio_prog a1 a2 a3  
Ci sono 4 argomenti:  
./mio_prog  
a1  
a2  
a3
```

# Esercizio 1 – Hello Server

- Implementare un semplice **server TCP mono-processo** che fornisce un messaggio "Hello!" ai client che si collegano
  - Leggere la porta da linea di comando (opzionale)
- Implementare il relativo client
  - Il client si connette, riceve il messaggio, lo stampa ed esce
  - Leggere indirizzo e porta del server da linea di comando
- Note:
  - Gestire gli errori
  - Server e client conoscono la dimensione del messaggio
- Compilazione ed esecuzione (eseguire da terminale, nella cartella dove si trovano i file, questi comandi, assumendo che l'indirizzo del server sia 127.0.0.1)
  - `gcc -o server <nome_file_server.c>`
  - `gcc -o client <nome_file_client.c>`
  - `./server 1234`
  - `./client 127.0.0.1 1234`

# Leggere una riga intera

```
#define BUFFER_SIZE 1024

...

int main(int argc, char* argv[]){
    ...
    char buffer[BUFFER_SIZE];

    ...

    fgets(buffer, BUFFER_SIZE, stdin);
}
```

# Esercizio 2 – Echo Server

- Implementare un **server TCP mono-processo** che re-invia al mittente un messaggio ricevuto
- Implementare un client che di continuo:
  - Rimane in attesa di ricevere una stringa da tastiera
  - Se la stringa è "Bye" interrompe la connessione ed esce
  - Altrimenti invia la stringa, riceve la risposta e la stampa
- Il server continua a fare echo al client finché la connessione non si interrompe
- Server e client leggono/scrivono sempre 20 byte
  - Occhio al newline! Va rimosso
- Provare a connettersi con un secondo client mentre il primo viene servito. Cosa succede?